

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación · Ingeniería de Software

Desarrollo de Aplicaciones Web con Java

De los servlets a los frameworks — guía completa orientada a un proyecto real

Servlets, JSP, JSTL y EL; luego Spring Boot y Jakarta Server Faces (JPA, JTA)

Proyecto final: inicio de sesión, control de acceso, CRUD, reportes y reportes impresos

Asignatura: Aplicaciones Web (5.º semestre)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026–2027

Versiones de referencia (Java 25 LTS, Tomcat 11, Jakarta EE 11, Spring Boot 4) en la sección 2.

Contenido

1	Introducción	4
1.1	Qué es una aplicación web y el modelo petición–respuesta	4
1.2	El contenedor de servlets y la arquitectura por capas	4
1.3	Lo que aprenderás	4
2	Preparación del entorno	5
2.1	JDK, IntelliJ IDEA y Tomcat	5
2.2	Estructura de un proyecto web y empaquetado WAR	5
2.3	El pom.xml de una aplicación web	6
2.4	Versiones de referencia (verificadas a junio de 2026)	7
3	Servlets: el controlador del lado del servidor	9
3.1	Anatomía de un servlet y su ciclo de vida	9
3.2	Leer datos de la petición	10
4	JSP: la vista	10
4.1	Una JSP como destino del servlet	11
4.2	Por qué evitar el código Java en la JSP	11
5	EL: el lenguaje de expresiones	11
5.1	Acceso a propiedades de un objeto (JavaBean)	12
6	JSTL: lógica de presentación sin scriptlets	12
6.1	Las etiquetas del núcleo (core)	12
6.2	Condicionales y enlaces	13
7	Arquitectura por capas y MVC	13
7.1	Las capas y el flujo de una petición	14
7.2	El modelo: una clase JavaBean	14
8	Sesiones, inicio de sesión y control de acceso	15
8.1	La sesión	15
8.2	El inicio de sesión	15
8.3	El control de acceso con un filtro	16

9	CRUD completo	17
9.1	El DAO con sentencias preparadas	17
9.2	El servlet controlador	19
9.3	Las vistas: formulario	21
10	Reportes	21
10.1	La consulta de agregación en el DAO	22
10.2	La vista del reporte	22
11	Reportes impresos	22
11.1	Camino sencillo: estilos de impresión (@media print)	23
11.2	Camino profesional: generar el PDF en el servidor	23
12	El problema completo: Sistema de Gestión de Estudiantes	24
12.1	Cómo encajan las piezas	25
12.2	La estructura final del proyecto	25
12.3	El esquema de la base de datos	26
13	Por qué usar un framework	28
14	Spring Boot paso a paso	28
14.1	Paso 1: crear el proyecto y las dependencias	28
14.2	Paso 2: configurar la base de datos	29
14.3	Paso 3: la entidad (JPA)	30
14.4	Paso 4: el repositorio	30
14.5	Paso 5: el servicio	31
14.6	Paso 6: el controlador y la vista (CRUD)	31
14.7	Paso 7: inicio de sesión y control de acceso con Spring Security	33
15	Jakarta Server Faces (JSF) con JPA y JTA paso a paso	34
15.1	Paso 1: la entidad JPA y la unidad de persistencia	35
15.2	Paso 2: el bean gestionado (CDI) con JPA y JTA	35
15.3	Paso 3: la vista con componentes (Facelets)	36
15.4	Paso 4: inicio de sesión y control de acceso en JSF	37
16	Cierre: los tres enfoques comparados	38

16.1 Recomendación de aprendizaje	39
16.2 Glosario	39
16.3 Recursos	40

1 Introducción

Esta guía te enseña a construir aplicaciones web con Java de principio a fin. Empezamos con los componentes fundamentales —servlets, JSP, JSTL y el lenguaje de expresiones— programando con clases y organizando el código por capas; con ellos resolveremos, problema a problema, las piezas de una aplicación real. Después daremos el salto a los frameworks del lado del servidor: Spring Boot y Jakarta Server Faces, paso a paso.

El método de la guía es práctico: en cada parte se plantea primero el problema concreto que se va a resolver (un subproblema), luego se explica con detalle la técnica que lo resuelve y se muestra un ejemplo pequeño pero completo. Al final reuniremos todos esos subproblemas en el problema completo: un sistema con inicio de sesión, control de acceso a las páginas, CRUD, reportes y reportes impresos.

Cómo está organizada

La **Parte I** construye la aplicación con tecnología base de Java (servlets, JSP, JSTL, EL) para que entiendas qué ocurre por debajo. La **Parte II** rehace ese mismo tipo de aplicación con frameworks (Spring Boot y Jakarta Faces), que automatizan gran parte del trabajo. Dominar primero la base hace que el framework tenga sentido.

1.1 Qué es una aplicación web y el modelo petición–respuesta

Una aplicación web es un programa que se ejecuta en un servidor y al que las personas acceden mediante un navegador. El navegador (cliente) envía una petición HTTP y el servidor responde, normalmente, con una página HTML generada dinámicamente a partir de datos. Este intercambio se llama modelo petición–respuesta (request–response).

Un detalle esencial es que HTTP no tiene estado: el servidor no recuerda al usuario entre una petición y la siguiente. Para «recordarlo» —y así implementar un inicio de sesión— se usan las sesiones, que verás en la sección 8. Comprender este modelo es la base de todo lo que sigue.

1.2 El contenedor de servlets y la arquitectura por capas

El código Java del lado del servidor no se ejecuta solo: vive dentro de un «contenedor de servlets» como Apache Tomcat, que recibe las peticiones HTTP, las entrega a tu código y devuelve la respuesta. Tú escribes la lógica; el contenedor se encarga de la red, los hilos y el ciclo de vida.

Para que la aplicación crezca de forma ordenada, separaremos el código en capas: el **modelo** (clases que representan los datos), el **acceso a datos** (DAO, que habla con la base), el **servicio** (la lógica de negocio), el **controlador** (servlets que coordinan) y la **vista** (JSP que genera el HTML). Esta separación es el patrón Modelo–Vista–Controlador (MVC), y es la columna vertebral de la guía.

1.3 Lo que aprenderás

Al terminar la Parte I sabrás resolver, con tecnología base de Java, cada uno de estos problemas; al terminar la Parte II sabrás resolverlos con dos frameworks distintos. En concreto:

- Responder peticiones con servlets y generar vistas con JSP, JSTL y EL.
- Estructurar la aplicación por capas, programando con clases (JavaBeans, DAO, servicios).
- Implementar un inicio de sesión y proteger páginas con control de acceso.
- Construir un CRUD completo conectado a una base de datos.
- Producir reportes en pantalla y reportes impresos (PDF).
- Rehacer la aplicación con Spring Boot y con Jakarta Server Faces (JPA, JTA).

2 Preparación del entorno

Para ejecutar código Java que atienda peticiones del navegador necesitamos un servidor que lo aloje y un proyecto correctamente empaquetado. ¿Cómo preparamos ese entorno?

Antes de programar necesitas tres cosas: el JDK (para compilar y ejecutar Java), un contenedor de servlets (Tomcat, que ejecuta tu aplicación) y una forma de construir y empaquetar el proyecto (Maven). Esta sección los pone en marcha.

2.1 JDK, IntelliJ IDEA y Tomcat

Instala una versión LTS del JDK (21 o 25) y IntelliJ IDEA como entorno de desarrollo. El contenedor será Apache Tomcat 11, que implementa Jakarta EE 11 (el estándar actual de Java empresarial). Descarga Tomcat, descomprímelo y, en IntelliJ, configúralo como servidor de la aplicación.

Un punto importante de nomenclatura: desde Jakarta EE 9 los paquetes pasaron de `javax.*` a `jakarta.*`. Por eso en esta guía verás `jakarta.servlet`, `jakarta.servlet.http`, etc. Si encuentras ejemplos antiguos con `javax.servlet`, son de versiones anteriores y no funcionarán en Tomcat 11.

2.2 Estructura de un proyecto web y empaquetado WAR

Una aplicación web Java se empaqueta en un archivo WAR (Web Application Archive) que se despliega en Tomcat. El proyecto sigue una estructura estándar: el código en `src/main/java`, las páginas y la configuración en `src/main/webapp`. La carpeta `WEB-INF` es especial: su contenido no es accesible directamente desde el navegador, por lo que es el lugar seguro para las JSP que solo deben servirse a través de un servlet.

Estructura del proyecto web

```

1 proyecto/
2 |- pom.xml
3 |- src/main/java/com/uteq/...      # servlets, modelo, dao, servicio
4 |- src/main/webapp/
5   |- index.jsp                    # paginas publicas
6   |- estilos.css
7   |- WEB-INF/
8     |- web.xml                    # configuracion (opcional con anotaciones)
9     |- vistas/                    # JSP protegidas (no accesibles directo)

```

2.3 El pom.xml de una aplicación web

El `pom.xml` declara que el empaquetado es `war` e incluye las dependencias de la API de servlets y de JSTL. La API de servlets se marca como `provided` porque la aporta Tomcat en tiempo de ejecución; no debe incluirse dentro del WAR.

Crear en: `pom.xml` (raiz del proyecto)

pom.xml (aplicación web)

```

1 <project>
2   <modelVersion>4.0.0</modelVersion>
3   <groupId>com.uteq</groupId>
4   <artifactId>gestion-estudiantes</artifactId>
5   <version>1.0.0</version>
6   <packaging>war</packaging>
7   <properties>
8     <maven.compiler.release>21</maven.compiler.release>
9   </properties>
10  <dependencies>
11    <dependency>
12      <groupId>jakarta.servlet</groupId>
13      <artifactId>jakarta.servlet-api</artifactId>
14      <version>6.1.0</version>
15      <scope>provided</scope>
16    </dependency>
17    <dependency>      <!-- JSTL (implementacion para Jakarta EE) -->
18      <groupId>org.glassfish.web</groupId>
19      <artifactId>jakarta.servlet.jsp.jstl</artifactId>
20      <version>3.0.1</version>
21    </dependency>
22    <dependency>      <!-- conector de PostgreSQL -->
23      <groupId>org.postgresql</groupId>
24      <artifactId>postgresql</artifactId>
25      <version>42.7.4</version>
26    </dependency>
27  </dependencies>
28 </project>

```

2.4 Versiones de referencia (verificadas a junio de 2026)

Estas son las versiones vigentes de la tecnología usada en la guía. Si una herramienta usa todavía `javax.*`, es anterior a Jakarta EE 9 y no es compatible con este entorno.

Componente	Versión vigente (jun. 2026)
Java (LTS)	25 (actual) y 21 (anterior).
Apache Tomcat	11.0.x (implementa Jakarta EE 11 web).
Jakarta Servlet	6.1 (paquete jakarta.servlet).
Jakarta EE	11 (estandar actual).
Spring Boot	4.0.x (sobre Spring 7, Java 17+).
Jakarta Faces	4.1 (parte de Jakarta EE 11).
PostgreSQL	18.4 (base de datos de los ejemplos).

PARTE I

Tecnología base: servlets, JSP, JSTL y EL

3 Servlets: el controlador del lado del servidor

Cuando el navegador pide una dirección de nuestra aplicación, ¿cómo hacemos que un trozo de código Java se ejecute en el servidor y devuelva una respuesta hecha a medida?

La respuesta es el servlet. Un servlet es una clase Java que el contenedor instancia y ejecuta para atender peticiones HTTP. Es la pieza que recibe la petición, decide qué hacer y produce (o delega) la respuesta. En la arquitectura MVC, el servlet hace de controlador.

3.1 Anatomía de un servlet y su ciclo de vida

Un servlet hereda de `HttpServlet` y sobrescribe los métodos que atienden cada tipo de petición: `doGet` para las peticiones GET (abrir una página) y `doPost` para las POST (enviar un formulario). El contenedor crea el servlet una sola vez y reutiliza esa instancia para todas las peticiones, llamando al método adecuado en cada una.

La anotación `@WebServlet` asocia el servlet con una dirección (URL). Cuando alguien visita esa dirección, el contenedor ejecuta el método correspondiente. El siguiente es un servlet completo que responde con HTML.

Crear en: `src/main/java/com/uteq/web/HolaServlet.java`

HolaServlet.java

```

1 package com.uteq.web;
2
3 import jakarta.servlet.http.HttpServlet;
4 import jakarta.servlet.http.HttpServletRequest;
5 import jakarta.servlet.http.HttpServletResponse;
6 import jakarta.servlet.annotation.WebServlet;
7 import java.io.IOException;
8 import java.io.PrintWriter;
9
10 @WebServlet("/hola")           // responde en /hola
11 public class HolaServlet extends HttpServlet {
12     @Override
13     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
14         throws IOException {
15         resp.setContentType("text/html; charset=UTF-8");
16         PrintWriter out = resp.getWriter();
17         out.println("<!DOCTYPE html><html lang='es'><body>");
18         out.println("<h1>Hola desde un servlet</h1>");
19         out.println("</body></html>");
20     }
21 }
```

Al desplegar el proyecto en Tomcat y visitar `http://localhost:8080/gestion-estudiantes/hola`, el contenedor ejecuta `doGet` y el navegador recibe el HTML. Fíjate en `setContentType` con `charset=UTF-8`: es lo que asegura que las tildes se muestren bien.

3.2 Leer datos de la petición

El usuario llena un formulario y lo envía. ¿Cómo lee el servlet esos datos para procesarlos?

Los datos enviados por el navegador (los campos de un formulario o los parámetros de la URL) se leen con `request.getParameter("nombre")`, que devuelve el valor como texto. Como esos datos vienen del exterior, hay que validarlos siempre en el servidor antes de usarlos.

Crear en: `src/main/java/com/uteq/web/SaludarServlet.java`

SaludarServlet.java (lee un parámetro)

```

1 package com.uteq.web;
2
3 import jakarta.servlet.*;
4 import jakarta.servlet.http.*;
5 import jakarta.servlet.annotation.WebServlet;
6 import java.io.IOException;
7
8 @WebServlet("/saludar")
9 public class SaludarServlet extends HttpServlet {
10     @Override
11     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
12         throws ServletException, IOException {
13         String nombre = req.getParameter("nombre"); // dato del formulario
14         if (nombre == null || nombre.isBlank()) {
15             nombre = "invitado"; // validacion simple
16         }
17         req.setAttribute("nombre", nombre); // se lo pasamos a la vista
18         req.getRequestDispatcher("/WEB-INF/vistas/saludo.jsp")
19             .forward(req, resp); // delega en la JSP
20     }
21 }

```

Observa el final: el servlet no escribe HTML, sino que guarda el dato como atributo y reenvía (`forward`) la petición a una JSP. Esto es el patrón MVC en acción: el servlet controla, la JSP presenta. Lo veremos en detalle en las dos secciones siguientes.

4 JSP: la vista

Generar HTML escribiendo `out.println` dentro del servlet es tedioso y mezcla lógica con presentación. ¿Cómo separamos el diseño de la página del código Java?

La solución es JSP (JavaServer Pages): un archivo que es, en esencia, HTML con la posibilidad de insertar datos dinámicos. El contenedor lo convierte internamente en un servlet, pero tú escribes algo muy parecido a una página HTML normal, lo que mantiene la presentación separada de la lógica.

4.1 Una JSP como destino del servlet

La JSP recibe los datos que el servlet dejó como atributos y los muestra. Colocamos las JSP dentro de **WEB-INF/vistas** para que no se puedan abrir directamente desde el navegador: solo se sirven cuando un servlet hace **forward** hacia ellas. Esta es la JSP a la que reenviaba el servlet anterior.

Crear en: `src/main/webapp/WEB-INF/vistas/saludo.jsp`

saludo.jsp

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <!DOCTYPE html>
3 <html lang="es">
4 <head><meta charset="UTF-8"><title>Saludo</title></head>
5 <body>
6   <h1>Hola, ${nombre}</h1>   <!-- ${nombre} es el dato del servlet -->
7 </body>
8 </html>

```

La expresión `${nombre}` pertenece al lenguaje de expresiones (EL), que es el tema de la sección siguiente. Lo importante ahora es la idea: el servlet decide y prepara los datos; la JSP solo los presenta.

4.2 Por qué evitar el código Java en la JSP

Las versiones antiguas de JSP permitían incrustar código Java entre `<% ... %>` (los llamados «scriptlets»). Hoy se considera una mala práctica, porque vuelve a mezclar lógica y presentación, justo lo que queríamos evitar. La regla moderna es: en la JSP, nada de código Java; solo EL y JSTL.

Regla de oro de la vista

En una JSP bien hecha no hay scriptlets `<% %>`. Toda la lógica vive en el servlet y el servicio; la JSP solo muestra datos con EL (`${...}`) y los recorre o condiciona con etiquetas JSTL. Así la vista es legible para un diseñador y mantenible para el equipo.

5 EL: el lenguaje de expresiones

Dentro de la JSP necesitamos leer datos (un texto, un objeto, una propiedad de ese objeto) sin escribir código Java. ¿Cómo lo hacemos de forma limpia?

EL (Expression Language) es un mini-lenguaje para acceder a datos dentro de la JSP con la sintaxis `${...}`. Puede leer atributos de la petición, de la sesión o de la aplicación, y navegar por las propiedades de los objetos llamando automáticamente a sus métodos `get`. Es lo que hace posible la regla «sin código Java en la vista».

5.1 Acceso a propiedades de un objeto (JavaBean)

Para que EL pueda leer las propiedades de un objeto, ese objeto debe ser un JavaBean: una clase con atributos privados y métodos `get/set` públicos. Cuando escribes `${estudiante.nombre}`, EL llama por dentro a `estudiante.getNombre()`. Por eso en esta guía el modelo se programa con clases JavaBean, y no con `record`.

Acceso con EL en una JSP

```
1 <!-- El servlet hizo: request.setAttribute("estudiante", e); -->
2 <p>Nombre: ${estudiante.nombre}</p>      <!-- llama a getNombre() -->
3 <p>Correo: ${estudiante.correo}</p>      <!-- llama a getCorreo() -->
4 <p>Suma: ${2 + 3}</p>                    <!-- EL tambien evalua expresiones -->
5 <p>Usuario: ${sessionScope.usuario}</p> <!-- dato guardado en la sesion -->
```

EL distingue varios «ámbitos» (scopes): `requestScope`, `sessionScope` y `applicationScope`. Si no indicas el ámbito, EL los busca en orden. Esta capacidad de leer la sesión será clave para mostrar quién ha iniciado sesión.

6 JSTL: lógica de presentación sin scriptlets

En la vista necesitamos repetir filas (una por cada estudiante de una lista) y mostrar contenido condicional, pero sin volver a escribir código Java. ¿Cómo?

JSTL (JavaServer Pages Standard Tag Library) es una biblioteca de etiquetas que aporta a la JSP las estructuras de control que necesita —bucles, condicionales, formato— en forma de etiquetas, no de código Java. Combinada con EL, cubre prácticamente todo lo que una vista necesita hacer.

6.1 Las etiquetas del núcleo (core)

Para usar JSTL se declara la biblioteca con una directiva `taglib` al inicio de la JSP. En Jakarta EE la URI del núcleo es `jakarta.tags.core`. Las etiquetas más usadas son `c:forEach` (repetir), `c:if` y `c:choose` (condicionar) y `c:out` (mostrar texto de forma segura).

Crear en: `src/main/webapp/WEB-INF/vistas/lista.jsp`

Lista con `c:forEach` (`lista.jsp`)

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
3 <table border="1">
4   <tr><th>ID</th><th>Nombre</th><th>Carrera</th></tr>
5   <c:forEach var="e" items="${estudiantes}">
6     <tr>
7       <td>${e.id}</td>
8       <td><c:out value="${e.nombre}"/></td>    <!-- c:out escapa: anti-XSS -->
9       <td><c:out value="${e.carrera}"/></td>
10    </tr>
11  </c:forEach>
12 </table>

```

La etiqueta `c:out` no solo muestra el valor: lo «escapa», convirtiendo caracteres como `<` o `>` en entidades. Eso evita el ataque XSS (inyección de scripts), igual que el escapado que aplicabas a mano en otros lenguajes. Es una buena práctica usar `c:out` para todo dato que provenga del usuario.

6.2 Condicionales y enlaces

Para mostrar algo solo si se cumple una condición se usa `c:if`; para elegir entre varias opciones, `c:choose`. Y para construir direcciones correctas dentro de la aplicación —respetando la ruta del contexto— se usa `c:url`, que evita errores cuando la aplicación cambia de nombre o de ubicación.

Condicional y enlaces JSTL

```

1 <c:if test="${not empty sessionScope.usuario}">
2   <p>Sesion de: <c:out value="${sessionScope.usuario}"/></p>
3 </c:if>
4
5 <c:choose>
6   <c:when test="${empty estudiantes}">No hay registros.</c:when>
7   <c:otherwise>Total: ${estudiantes.size()}</c:otherwise>
8 </c:choose>
9
10 <a href="<c:url value='/estudiantes?accion=nuevo'/">">Nuevo</a>

```

7 Arquitectura por capas y MVC

Con servlets y JSP ya podemos responder peticiones, pero si mezclamos el acceso a datos, la lógica y la presentación en cada servlet, el proyecto se vuelve inmanejable. ¿Cómo lo organizamos para que crezca sano?

La solución es separar responsabilidades en capas y aplicar el patrón Modelo–Vista–Controlador. Cada capa tiene un único propósito y solo conoce a la capa que tiene justo debajo. Así el código se entiende, se prueba y se modifica sin efectos colaterales.

7.1 Las capas y el flujo de una petición

Estas son las capas que usaremos en toda la Parte I. Programar con clases bien definidas en cada capa es lo que mantiene el orden a medida que la aplicación crece.

Capa	Responsabilidad
Modelo	Clases JavaBean que representan los datos (p. ej. Estudiante).
DAO	Acceso a datos: traduce entre objetos y la base (JDBC).
Servicio	Logica de negocio y validaciones; coordina los DAO.
Controlador	Servlets: reciben la petición y deciden que hacer.
Vista	JSP con JSTL y EL: presentan el resultado.

El flujo de una petición es siempre el mismo: el navegador llama a un servlet (controlador); el servlet pide datos al servicio; el servicio usa el DAO para hablar con la base; el servlet guarda el resultado como atributo y reenvía a una JSP (vista); la JSP genera el HTML con EL y JSTL. Mantener este flujo es la clave de un proyecto ordenado.

7.2 El modelo: una clase JavaBean

El modelo es una clase con atributos privados y métodos `get/set`, tal como exige EL. Representa una fila de la tabla `estudiantes`. Esta clase se usará en toda la aplicación.

Crear en: `src/main/java/com/uteq/modelo/Estudiante.java`

`modelo/Estudiante.java`

```

1 package com.uteq.modelo;
2
3 public class Estudiante {
4     private int id;
5     private String nombre;
6     private String correo;
7     private String carrera;
8
9     public Estudiante() {}
10    public Estudiante(int id, String nombre, String correo, String carrera) {
11        this.id = id; this.nombre = nombre;
12        this.correo = correo; this.carrera = carrera;
13    }
14    public int getId() { return id; }
15    public void setId(int id) { this.id = id; }
16    public String getNombre() { return nombre; }
17    public void setNombre(String nombre) { this.nombre = nombre; }
18    public String getCorreo() { return correo; }
19    public void setCorreo(String correo) { this.correo = correo; }
20    public String getCarrera() { return carrera; }
21    public void setCarrera(String carrera) { this.carrera = carrera; }
22 }

```

8 Sesiones, inicio de sesión y control de acceso

Como HTTP no recuerda al usuario, necesitamos: (1) un inicio de sesión que verifique sus credenciales y, una vez dentro, (2) un mecanismo que impida el acceso a las páginas privadas a quien no ha iniciado sesión. ¿Cómo se construyen ambos?

La pieza que mantiene el estado del usuario es la sesión (`HttpSession`): un espacio de almacenamiento en el servidor, asociado a cada visitante, donde guardamos por ejemplo su nombre tras iniciar sesión. Con la sesión resolvemos el primer problema; con un «filtro», el segundo.

8.1 La sesión

La sesión se obtiene con `request.getSession()`. En ella se guardan datos con `setAttribute` y se leen con `getAttribute`. Mientras exista un dato que marque al usuario como autenticado, sabremos que ha iniciado sesión. Para cerrar la sesión se llama a `invalidate()`.

8.2 El inicio de sesión

El servlet de login recibe el usuario y la contraseña, los verifica y, si son correctos, guarda al usuario en la sesión y lo redirige a la zona privada. Las contraseñas nunca se guardan en texto plano:

se almacena un «hash» (con BCrypt, mediante una librería) y se compara con `checkpw`. Aquí mostramos la estructura del servlet.

Crear en: `src/main/java/com/uteq/web/LoginServlet.java`

LoginServlet.java

```

1 package com.uteq.web;
2
3 import com.uteq.servicio.UsuarioServicio;
4 import jakarta.servlet.*;
5 import jakarta.servlet.http.*;
6 import jakarta.servlet.annotation.WebServlet;
7 import java.io.IOException;
8
9 @WebServlet("/login")
10 public class LoginServlet extends HttpServlet {
11     private final UsuarioServicio servicio = new UsuarioServicio();
12
13     @Override
14     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
15         throws ServletException, IOException {
16         req.getRequestDispatcher("/WEB-INF/vistas/login.jsp").forward(req, resp);
17     }
18
19     @Override
20     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
21         throws ServletException, IOException {
22         String usuario = req.getParameter("usuario");
23         String clave = req.getParameter("clave");
24         if (servicio.credencialesValidas(usuario, clave)) {
25             HttpSession session = req.getSession();
26             session.setAttribute("usuario", usuario); // queda autenticado
27             resp.sendRedirect(req.getContextPath() + "/estudiantes");
28         } else {
29             req.setAttribute("error", "Credenciales incorrectas");
30             req.getRequestDispatcher("/WEB-INF/vistas/login.jsp").forward(req, resp);
31         }
32     }
33 }

```

8.3 El control de acceso con un filtro

Para proteger todas las páginas privadas de una vez —sin repetir la comprobación en cada servlet— se usa un filtro (**Filter**). Un filtro intercepta las peticiones antes de que lleguen a su destino. Nuestro filtro deja pasar solo si hay un usuario en la sesión; en caso contrario, redirige al login. La anotación `@WebFilter` indica qué rutas protege.

Crear en: `src/main/java/com/uteq/web/AutenticacionFilter.java`

AutenticacionFilter.java

```

1 package com.uteq.web;
2
3 import jakarta.servlet.*;
4 import jakarta.servlet.http.*;
5 import jakarta.servlet.annotation.WebFilter;
6 import java.io.IOException;
7
8 @WebFilter("/estudiantes") // protege la zona privada
9 public class AutenticacionFilter implements Filter {
10     @Override
11     public void doFilter(ServletRequest req, ServletResponse resp,
12         FilterChain chain) throws IOException, ServletException {
13         HttpServletRequest r = (HttpServletRequest) req;
14         HttpServletResponse w = (HttpServletResponse) resp;
15         HttpSession session = r.getSession(false);
16         boolean autenticado = session != null
17             && session.getAttribute("usuario") != null;
18         if (autenticado) {
19             chain.doFilter(req, resp); // continua hacia el servlet
20         } else {
21             w.sendRedirect(r.getContextPath() + "/login"); // bloquea
22         }
23     }
24 }

```

Por qué un filtro y no comprobar en cada servlet

Centralizar el control de acceso en un filtro evita repetir la comprobación en todos los servlets (y olvidarla en alguno, que es justo donde aparecen los fallos de seguridad). Un único punto de control es más seguro y más fácil de mantener.

9 CRUD completo

El corazón de casi toda aplicación de gestión es poder crear, listar, editar y eliminar registros. ¿Cómo implementamos ese CRUD con servlets, un DAO y vistas JSP?

Resolveremos el CRUD repartiendo el trabajo entre las capas: un DAO que ejecuta el SQL con sentencias preparadas, un servlet controlador que enruta las acciones (listar, nuevo, guardar, editar, eliminar) y unas JSP que muestran la tabla y el formulario. Empezamos por el DAO.

9.1 El DAO con sentencias preparadas

El DAO concentra todo el acceso a datos. Cada operación usa un `PreparedStatement` con marcadores `?`, la defensa contra la inyección SQL. Abrimos la conexión dentro de un

try-with-resources para que se cierre sola. Por su longitud, lo mostramos en dos partes.

Crear en: src/main/java/com/uteq/dao/EstudianteDAO.java

dao/EstudianteDAO.java — parte 1/2

```
1 package com.uteq.dao;
2
3 import com.uteq.modelo.Estudiante;
4 import java.sql.*;
5 import java.util.*;
6
7 public class EstudianteDAO {
8     private static final String URL =
9         "jdbc:postgresql://127.0.0.1:5432/appweb2026ppa";
10    private static final String USER = "postgres";
11    private static final String PASS = "P@$7Gr3$QL";
12
13    private Connection conectar() throws SQLException {
14        return DriverManager.getConnection(URL, USER, PASS);
15    }
16
17    public List<Estudiante> listar() throws SQLException {
18        List<Estudiante> lista = new ArrayList<>();
19        String sql = "SELECT id, nombre, correo, carrera "
20            + "FROM estudiantes ORDER BY id";
21        try (Connection con = conectar();
22            Statement st = con.createStatement();
23            ResultSet rs = st.executeQuery(sql)) {
24            while (rs.next()) {
25                lista.add(new Estudiante(rs.getInt("id"),
26                    rs.getString("nombre"), rs.getString("correo"),
27                    rs.getString("carrera")));
28            }
29        }
30        return lista;
31    }
32
33    public Estudiante buscar(int id) throws SQLException {
34        String sql = "SELECT id, nombre, correo, carrera "
35            + "FROM estudiantes WHERE id = ?";
36        try (Connection con = conectar();
37            PreparedStatement ps = con.prepareStatement(sql)) {
38            ps.setInt(1, id);
39            try (ResultSet rs = ps.executeQuery()) {
40                if (rs.next()) {
41                    return new Estudiante(rs.getInt("id"),
42                        rs.getString("nombre"), rs.getString("correo"),
43                        rs.getString("carrera"));
44                }
45            }
46        }
47    }
48 }
```

dao/EstudianteDAO.java — parte 2/2

```

1      return null;
2  }

```

Crear en: src/main/java/com/uteq/dao/EstudianteDAO.java

dao/EstudianteDAO.java (continuación)

```

1  public void crear(Estudiante e) throws SQLException {
2      String sql = "INSERT INTO estudiantes (nombre, correo, carrera) "
3                  + "VALUES (?, ?, ?)";
4      try (Connection con = conectar();
5           PreparedStatement ps = con.prepareStatement(sql)) {
6          ps.setString(1, e.getNombre());
7          ps.setString(2, e.getCorreo());
8          ps.setString(3, e.getCarrera());
9          ps.executeUpdate();
10     }
11 }
12
13 public void actualizar(Estudiante e) throws SQLException {
14     String sql = "UPDATE estudiantes SET nombre=?, correo=?, "
15                 + "carrera=? WHERE id=?";
16     try (Connection con = conectar();
17          PreparedStatement ps = con.prepareStatement(sql)) {
18         ps.setString(1, e.getNombre());
19         ps.setString(2, e.getCorreo());
20         ps.setString(3, e.getCarrera());
21         ps.setInt(4, e.getId());
22         ps.executeUpdate();
23     }
24 }
25
26 public void eliminar(int id) throws SQLException {
27     try (Connection con = conectar();
28          PreparedStatement ps = con.prepareStatement(
29              "DELETE FROM estudiantes WHERE id = ?")) {
30         ps.setInt(1, id);
31         ps.executeUpdate();
32     }
33 }
34 }

```

9.2 El servlet controlador

Un único servlet atiende todas las acciones del CRUD usando un parámetro `accion` que indica qué hacer. Tras una operación que modifica datos (guardar o eliminar) se redirige al listado: es el patrón Post-Redirect-Get, que evita que recargar la página repita la operación.

Crear en: src/main/java/com/uteq/web/EstudianteServlet.java

web/EstudianteServlet.java — parte 1/2

```

1 package com.uteq.web;
2
3 import com.uteq.dao.EstudianteDAO;
4 import com.uteq.modelo.Estudiante;
5 import jakarta.servlet.*;
6 import jakarta.servlet.http.*;
7 import jakarta.servlet.annotation.WebServlet;
8 import java.io.IOException;
9
10 @WebServlet("/estudiantes")
11 public class EstudianteServlet extends HttpServlet {
12     private final EstudianteDAO dao = new EstudianteDAO();
13
14     @Override
15     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
16         throws ServletException, IOException {
17         String accion = req.getParameter("accion");
18         try {
19             if ("nuevo".equals(accion) || "editar".equals(accion)) {
20                 if ("editar".equals(accion)) {
21                     int id = Integer.parseInt(req.getParameter("id"));
22                     req.setAttribute("estudiante", dao.buscar(id));
23                 }
24                 req.getRequestDispatcher("/WEB-INF/vistas/form.jsp")
25                     .forward(req, resp);
26             } else if ("eliminar".equals(accion)) {
27                 dao.eliminar(Integer.parseInt(req.getParameter("id")));
28                 resp.sendRedirect(req.getContextPath() + "/estudiantes");
29             } else {
30                 req.setAttribute("estudiantes", dao.listar());
31                 req.getRequestDispatcher("/WEB-INF/vistas/lista.jsp")
32                     .forward(req, resp);
33             }
34         } catch (Exception ex) { throw new ServletException(ex); }
35     }
36
37     @Override
38     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
39         throws ServletException, IOException {
40         Estudiante e = new Estudiante();
41         String id = req.getParameter("id");
42         e.setNombre(req.getParameter("nombre"));
43         e.setCorreo(req.getParameter("correo"));
44         e.setCarrera(req.getParameter("carrera"));
45         try {
46             if (id == null || id.isBlank()) {

```

web/EstudianteServlet.java — parte 2/2

```

1         dao.crear(e);
2     } else {
3         e.setId(Integer.parseInt(id));
4         dao.actualizar(e);
5     }
6     } catch (Exception ex) { throw new ServletException(ex); }
7     resp.sendRedirect(req.getContextPath() + "/estudiantes"); // PRG
8 }
9 }

```

9.3 Las vistas: formulario

La JSP del formulario sirve tanto para crear como para editar. Si recibe un estudiante (al editar), rellena los campos con sus datos y añade un campo oculto con el `id`; si no, los deja vacíos para una alta. El listado ya lo viste en la sección 6 con `c:forEach`.

Crear en: `src/main/webapp/WEB-INF/vistas/form.jsp`

form.jsp

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
3 <!DOCTYPE html><html lang="es"><head><meta charset="UTF-8">
4 <title>Estudiante</title></head><body>
5     <h1>${empty estudiante ? 'Nuevo' : 'Editar'} estudiante</h1>
6     <form method="post" action="<c:url value='/estudiantes' />">
7         <input type="hidden" name="id" value="${estudiante.id}">
8         <input name="nombre" value="${estudiante.nombre}" required>
9         <input name="correo" value="${estudiante.correo}" required type="email">
10        <input name="carrera" value="${estudiante.carrera}" required>
11        <button type="submit">Guardar</button>
12    </form>
13 </body></html>

```

10 Reportes

Más allá de listar registros, la dirección necesita información resumida: por ejemplo, cuántos estudiantes hay por carrera. ¿Cómo generamos un reporte así?

Un reporte se obtiene con una consulta de agregación (`GROUP BY`) en el DAO y se presenta con una JSP, igual que el listado. La diferencia está en la consulta, que en lugar de filas individuales devuelve totales agrupados.

10.1 La consulta de agregación en el DAO

Añadimos al DAO un método que cuente estudiantes por carrera. Devolvemos los resultados en un mapa ordenado de carrera a cantidad, que la vista recorrerá.

Método de reporte (en EstudianteDAO)

```

1 public Map<String, Integer> contarPorCarrera() throws SQLException {
2     Map<String, Integer> r = new LinkedHashMap<>();
3     String sql = "SELECT carrera, COUNT(*) AS total "
4         + "FROM estudiantes GROUP BY carrera ORDER BY carrera";
5     try (Connection con = conectar();
6         Statement st = con.createStatement();
7         ResultSet rs = st.executeQuery(sql)) {
8         while (rs.next()) {
9             r.put(rs.getString("carrera"), rs.getInt("total"));
10        }
11    }
12    return r;
13 }

```

10.2 La vista del reporte

El servlet de reporte llama a ese método y reenvía a una JSP que recorre el mapa con `c:forEach`. Para un mapa, la variable de iteración expone `key` (la carrera) y `value` (el total).

Crear en: `src/main/webapp/WEB-INF/vistas/reporte.jsp`

reporte.jsp

```

1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
3 <!DOCTYPE html><html lang="es"><head><meta charset="UTF-8">
4 <title>Reporte por carrera</title></head><body>
5     <h1>Estudiantes por carrera</h1>
6     <table border="1">
7         <tr><th>Carrera</th><th>Total</th></tr>
8         <c:forEach var="fila" items="${porCarrera}">
9             <tr><td><c:out value="${fila.key}" /></td><td>${fila.value}</td></tr>
10        </c:forEach>
11    </table>
12    <a href="<c:url value='/reporte?formato=pdf' />">Imprimir (PDF)</a>
13 </body></html>

```

11 Reportes impresos

El reporte en pantalla está bien, pero la secretaría necesita un documento para imprimir o archivar, idealmente en PDF. ¿Cómo lo generamos?

Hay dos caminos, y conviene conocer ambos. El primero, muy simple, es preparar la página para que el navegador la imprima bien usando estilos de impresión. El segundo, más profesional, es generar un PDF en el servidor con una librería. Veremos los dos.

11.1 Camino sencillo: estilos de impresión (@media print)

Con una hoja de estilos para impresión puedes ocultar lo que no debe imprimirse (menús, botones) y ajustar el formato. El navegador hace el resto al imprimir o «guardar como PDF». Es la vía más rápida y no requiere librerías.

Estilos de impresión (en la JSP)

```
1 <style>
2   @media print {
3     nav, .botones, a { display: none; }    /* ocultar al imprimir */
4     body { font-size: 12pt; }
5     table { width: 100%; border-collapse: collapse; }
6   }
7 </style>
8 <!-- El usuario imprime con Ctrl+P y elige 'Guardar como PDF' -->
```

11.2 Camino profesional: generar el PDF en el servidor

Para un PDF generado por el servidor (con formato fijo, listo para archivar) se usa una librería como OpenPDF. El servlet fija el tipo de contenido a `application/pdf`, construye el documento y lo escribe en el flujo de salida de la respuesta. Así el navegador recibe directamente el archivo.

Crear en: `src/main/java/com/uteq/web/ReportePdfServlet.java`

ReportePdfServlet.java (con OpenPDF)

```

1 package com.uteq.web;
2
3 import com.lowagie.text.*;
4 import com.lowagie.text.pdf.PdfWriter;
5 import com.uteq.dao.EstudianteDAO;
6 import jakarta.servlet.http.*;
7 import jakarta.servlet.annotation.WebServlet;
8 import java.util.Map;
9
10 @WebServlet("/reporte-pdf")
11 public class ReportePdfServlet extends HttpServlet {
12     private final EstudianteDAO dao = new EstudianteDAO();
13
14     @Override
15     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
16         throws java.io.IOException {
17         resp.setContentType("application/pdf");
18         resp.setHeader("Content-Disposition",
19             "attachment; filename=reporte.pdf");
20         try {
21             Map<String,Integer> datos = dao.contarPorCarrera();
22             Document doc = new Document();
23             PdfWriter.getInstance(doc, resp.getOutputStream());
24             doc.open();
25             doc.add(new Paragraph("Estudiantes por carrera"));
26             datos.forEach((carrera, total) ->
27                 doc.add(new Paragraph(carrera + ": " + total)));
28             doc.close();
29         } catch (Exception e) {
30             throw new RuntimeException(e);
31         }
32     }
33 }

```

La dependencia de OpenPDF

Para este servlet añada al `pom.xml` la dependencia `com.github.librepdf:openpdf`. OpenPDF es una librería libre y madura para generar PDF en Java; alternativas conocidas son iText y, para reportes complejos con plantillas, JasperReports.

12 El problema completo: Sistema de Gestión de Estudiantes

Hemos resuelto cada subproblema por separado. Ahora los reunimos en el problema completo, que es el objetivo de toda la Parte I. Lee primero el enunciado y luego cómo encajan las piezas que ya construiste.

Construir un Sistema de Gestión de Estudiantes con los siguientes requisitos:

- 1) **Inicio de sesión:** una página de acceso que valide usuario y contraseña.
- 2) **Control de acceso:** las páginas de gestión solo son accesibles tras iniciar sesión; quien no lo haya hecho es redirigido al login.
- 3) **CRUD:** crear, listar, editar y eliminar estudiantes (nombre, correo, carrera).
- 4) **Reportes:** una vista con el total de estudiantes por carrera.
- 5) **Reportes impresos:** poder descargar ese reporte en PDF.

12.1 Cómo encajan las piezas

La solución es la suma ordenada de lo construido: el modelo y el DAO gestionan los datos; el servicio valida; los servlets controlan; el filtro protege; y las JSP con JSTL y EL presentan. El siguiente mapa relaciona cada requisito con las piezas que lo resuelven.

Requisito	Piezas que lo resuelven
Inicio de sesión	LoginServlet + login.jsp + UsuarioServicio (hash BCrypt).
Control de acceso	AutenticacionFilter sobre las rutas privadas + la sesión.
CRUD	EstudianteServlet + EstudianteDAO + lista.jsp y form.jsp.
Reportes	contarPorCarrera() + reporte.jsp con c:forEach.
Reportes impresos	@media print y ReportePdfServlet (OpenPDF).

12.2 La estructura final del proyecto

Reunidas todas las clases y vistas, el proyecto queda así. Cada archivo es uno de los que ya programaste en las secciones anteriores; aquí solo se ve el conjunto.

Estructura del sistema completo

```

1 gestion-estudiantes/
2 |- pom.xml
3 |- src/main/java/com/uteq/
4 |   |- modelo/Estudiante.java
5 |   |- dao/EstudianteDAO.java           # listar, buscar, crear, actualizar,
6 |   |                                   # eliminar, contarPorCarrera
7 |   |- servicio/UsuarioServicio.java   # credencialesValidas (hash)
8 |   |- web/LoginServlet.java
9 |   |- web/AutenticacionFilter.java
10 |   |- web/EstudianteServlet.java
11 |   |- web/ReportePdfServlet.java
12 |- src/main/webapp/
13 |   |- index.jsp
14 |   |- WEB-INF/vistas/
15 |       |- login.jsp  lista.jsp  form.jsp  reporte.jsp

```

12.3 El esquema de la base de datos

Para completar, este es el SQL de las dos tablas que usa el sistema. La contraseña se guarda como hash, nunca en claro.

Crear en: src/main/resources/db/schema.sql

schema.sql

```

1 CREATE TABLE IF NOT EXISTS estudiantes (
2     id          SERIAL          PRIMARY KEY,
3     nombre     VARCHAR(100) NOT NULL,
4     correo     VARCHAR(120) NOT NULL,
5     carrera    VARCHAR(80)  NOT NULL
6 );
7
8 CREATE TABLE IF NOT EXISTS usuarios (
9     id          SERIAL          PRIMARY KEY,
10    usuario     VARCHAR(50)  NOT NULL UNIQUE,
11    clave       VARCHAR(100) NOT NULL    -- hash BCrypt, no texto plano
12 );

```

Con esto, la Parte I queda completa: tienes una aplicación web funcional construida con la tecnología base de Java, entendiendo cada pieza. En la Parte II resolveremos el mismo problema con frameworks que automatizan gran parte de este trabajo.

PARTE II

Frameworks del backend: Spring Boot y Jakarta Faces

13 Por qué usar un framework

En la Parte I escribiste a mano la conexión, el enrutamiento, el control de acceso y el acceso a datos. En proyectos grandes eso es mucho código repetitivo y propenso a errores. ¿Cómo lo evitamos?

Un framework aporta resueltos los problemas comunes —enrutamiento, acceso a datos con un ORM, seguridad, transacciones— para que te centres en la lógica propia de tu aplicación. Veremos dos enfoques muy distintos y muy usados: Spring Boot, basado en la inyección de dependencias y la autoconfiguración; y Jakarta Server Faces, basado en componentes, dentro del estándar Jakarta EE.

La recompensa de haber hecho la Parte I a mano es que ahora reconocerás, bajo cada función del framework, el concepto que ya dominas: el controlador, la vista, el acceso a datos, la sesión, la transacción.

14 Spring Boot paso a paso

Construir el mismo tipo de aplicación (entidades persistentes, CRUD, seguridad con login y control de acceso) pero con mucho menos código repetitivo y con configuración mínima. ¿Cómo lo hace Spring Boot?

Spring Boot parte de un proyecto autoconfigurado: declaras las dependencias («starters») y el framework configura por ti el servidor, el acceso a datos y la seguridad. Usa la inyección de dependencias para conectar las capas, un ORM (JPA/Hibernate) para los datos y Spring Security para el inicio de sesión y el control de acceso. Lo construimos por pasos.

14.1 Paso 1: crear el proyecto y las dependencias

El proyecto se genera en Spring Initializr (start.spring.io) eligiendo los starters. Para nuestra aplicación necesitamos web (MVC y el motor de plantillas), datos (JPA), el conector de PostgreSQL, seguridad y Thymeleaf como vista. Estas son las dependencias clave del `pom.xml`.

Crear en: pom.xml (raíz del proyecto)

Dependencias de Spring Boot (pom.xml)

```

1 <parent>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-parent</artifactId>
4   <version>4.0.6</version>
5 </parent>
6 <dependencies>
7   <dependency>
8     <groupId>org.springframework.boot</groupId>
9     <artifactId>spring-boot-starter-web</artifactId>
10  </dependency>
11  <dependency>
12    <groupId>org.springframework.boot</groupId>
13    <artifactId>spring-boot-starter-data-jpa</artifactId>
14  </dependency>
15  <dependency>
16    <groupId>org.springframework.boot</groupId>
17    <artifactId>spring-boot-starter-thymeleaf</artifactId>
18  </dependency>
19  <dependency>
20    <groupId>org.springframework.boot</groupId>
21    <artifactId>spring-boot-starter-security</artifactId>
22  </dependency>
23  <dependency>
24    <groupId>org.postgresql</groupId>
25    <artifactId>postgresql</artifactId>
26    <scope>runtime</scope>
27  </dependency>
28 </dependencies>

```

14.2 Paso 2: configurar la base de datos

La conexión no se programa: se declara en `application.properties`. Spring Boot crea por ti el origen de datos y el gestor de persistencia a partir de estas líneas. La opción `ddl-auto` puede crear o actualizar las tablas a partir de las entidades.

Crear en: src/main/resources/application.properties

application.properties

```

1 spring.datasource.url=jdbc:postgresql://127.0.0.1:5432/appweb2026ppa
2 spring.datasource.username=postgres
3 spring.datasource.password=P@$7Gr3$QL
4 spring.jpa.hibernate.ddl-auto=update
5 spring.jpa.show-sql=true

```

14.3 Paso 3: la entidad (JPA)

Con JPA, una clase anotada con `@Entity` se corresponde con una tabla, y cada objeto con una fila; no escribimos SQL. Las anotaciones indican la clave primaria y su generación. Esta es la entidad equivalente al modelo de la Parte I.

Crear en: `src/main/java/com/uteq/modelo/Estudiante.java`

Estudiante.java (entidad JPA)

```
1 package com.uteq.modelo;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "estudiantes")
7 public class Estudiante {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11    private String nombre;
12    private String correo;
13    private String carrera;
14
15    // getters y setters (necesarios para JPA y para la vista)
16    public Long getId() { return id; }
17    public void setId(Long id) { this.id = id; }
18    public String getNombre() { return nombre; }
19    public void setNombre(String nombre) { this.nombre = nombre; }
20    public String getCorreo() { return correo; }
21    public void setCorreo(String correo) { this.correo = correo; }
22    public String getCarrera() { return carrera; }
23    public void setCarrera(String carrera) { this.carrera = carrera; }
24 }
```

14.4 Paso 4: el repositorio

Aquí está la magia de Spring Data JPA: defines una interfaz que extiende `JpaRepository` y obtienes gratis los métodos para listar, buscar, guardar y eliminar. No escribes la implementación: Spring la genera. Incluso puedes declarar consultas por el nombre del método.

Crear en: `src/main/java/com/uteq/repositorio/EstudianteRepositorio.java`

EstudianteRepositorio.java

```

1 package com.uteq.repositorio;
2
3 import com.uteq.modelo.Estudiante;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import java.util.List;
6
7 public interface EstudianteRepositorio
8     extends JpaRepository<Estudiante, Long> {
9     // Consulta derivada del nombre del metodo (sin SQL):
10    List<Estudiante> findByCarrera(String carrera);
11 }

```

14.5 Paso 5: el servicio

El servicio contiene la lógica de negocio y se apoya en el repositorio, que recibe por inyección de dependencias (Spring lo provee automáticamente). La anotación `@Service` lo registra como componente gestionado por el framework.

Crear en: `src/main/java/com/uteq/servicio/EstudianteServicio.java`

EstudianteServicio.java

```

1 package com.uteq.servicio;
2
3 import com.uteq.modelo.Estudiante;
4 import com.uteq.repositorio.EstudianteRepositorio;
5 import org.springframework.stereotype.Service;
6 import java.util.List;
7
8 @Service
9 public class EstudianteServicio {
10     private final EstudianteRepositorio repo;
11     public EstudianteServicio(EstudianteRepositorio repo) { // inyeccion
12         this.repo = repo;
13     }
14     public List<Estudiante> listar() { return repo.findAll(); }
15     public Estudiante buscar(Long id) { return repo.findById(id).orElse(null); }
16     public void guardar(Estudiante e) { repo.save(e); } // crea o actualiza
17     public void eliminar(Long id) { repo.deleteById(id); }
18 }

```

14.6 Paso 6: el controlador y la vista (CRUD)

El controlador (`@Controller`) mapea las rutas a métodos con `@GetMapping` y `@PostMapping`, pide datos al servicio y devuelve el nombre de una plantilla Thymeleaf, pasándole los datos en el `Model`. Compáralo con el `EstudianteServlet` de la Parte I: hace lo mismo, con mucho menos código.

Crear en: `src/main/java/com/uteq/controlador/EstudianteControlador.java`

EstudianteControlador.java

```

1 package com.uteq.controlador;
2
3 import com.uteq.modelo.Estudiante;
4 import com.uteq.servicio.EstudianteServicio;
5 import org.springframework.stereotype.Controller;
6 import org.springframework.ui.Model;
7 import org.springframework.web.bind.annotation.*;
8
9 @Controller
10 @RequestMapping("/estudiantes")
11 public class EstudianteControlador {
12     private final EstudianteServicio servicio;
13     public EstudianteControlador(EstudianteServicio servicio) {
14         this.servicio = servicio;
15     }
16
17     @GetMapping
18     public String listar(Model model) {
19         model.addAttribute("estudiantes", servicio.listar());
20         return "lista"; // plantilla lista.html
21     }
22
23     @GetMapping("/nuevo")
24     public String nuevo(Model model) {
25         model.addAttribute("estudiante", new Estudiante());
26         return "form";
27     }
28
29     @PostMapping("/guardar")
30     public String guardar(@ModelAttribute Estudiante estudiante) {
31         servicio.guardar(estudiante);
32         return "redirect:/estudiantes"; // patron PRG
33     }
34
35     @GetMapping("/eliminar/{id}")
36     public String eliminar(@PathVariable Long id) {
37         servicio.eliminar(id);
38         return "redirect:/estudiantes";
39     }
40 }

```

La vista es una plantilla Thymeleaf (`lista.html`). Thymeleaf es a Spring lo que JSTL/EL eran a la JSP: recorre y muestra datos con atributos especiales, escapando por defecto.

Crear en: `src/main/resources/templates/lista.html`

lista.html (Thymeleaf)

```
1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org" lang="es">
3 <head><meta charset="UTF-8"><title>Estudiantes</title></head>
4 <body>
5   <a th:href="@{/estudiantes/nuevo}">Nuevo</a>
6   <table>
7     <tr><th>Nombre</th><th>Carrera</th><th></th></tr>
8     <tr th:each="e : ${estudiantes}">
9       <td th:text="${e.nombre}">-</td>
10      <td th:text="${e.carrera}">-</td>
11      <td><a th:href="@{/estudiantes/eliminar/{id}(id=${e.id})}">Eliminar</a></td>
12    </tr>
13  </table>
14 </body>
15 </html>
```

14.7 Paso 7: inicio de sesión y control de acceso con Spring Security

Spring Security resuelve de una vez el login y el control de acceso que en la Parte I programaste con un servlet y un filtro. Defines una clase de configuración que declara qué rutas son públicas, cuáles requieren autenticación y cómo es el formulario de acceso. El framework se encarga del filtro, la sesión y el hash de contraseñas.

Crear en: `src/main/java/com/uteq/config/SeguridadConfig.java`

SeguridadConfig.java

```

1 package com.uteq.config;
2
3 import org.springframework.context.annotation.*;
4 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
5 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
6 import org.springframework.security.crypto.password.PasswordEncoder;
7 import org.springframework.security.web.SecurityFilterChain;
8
9 @Configuration
10 public class SeguridadConfig {
11     @Bean
12     public SecurityFilterChain filtros(HttpSecurity http) throws Exception {
13         http
14             .authorizeHttpRequests(a -> a
15                 .requestMatchers("/login", "/css/**").permitAll() // publico
16                 .anyRequest().authenticated() // resto protegido
17                 .formLogin(f -> f.loginPage("/login").permitAll());
18         return http.build();
19     }
20
21     @Bean
22     public PasswordEncoder passwordEncoder() {
23         return new BCryptPasswordEncoder(); // hash de contraseñas
24     }
25 }

```

Mira lo que desapareció

Compara con la Parte I: aquí no hay `AutenticacionFilter`, ni gestión manual de la sesión, ni código para conectar a la base, ni SQL del CRUD. Spring Boot lo aporta; tú declaras intenciones. Por eso era importante entender antes qué hace por debajo.

15 Jakarta Server Faces (JSF) con JPA y JTA paso a paso

Resolver el mismo problema, pero con un enfoque basado en componentes y dentro del estándar Jakarta EE, donde la vista se compone de componentes con estado y las transacciones las gestiona el contenedor. ¿Cómo se construye?

Jakarta Server Faces (JSF) es el framework de interfaces basado en componentes del estándar Jakarta EE. En lugar de servlets y plantillas, defines vistas con componentes (en archivos `Facelets.xhtml`) conectados a «beans» gestionados. La persistencia se hace con JPA y las transacciones con JTA, gestionadas por el contenedor (Payara, WildFly o Tomcat con las extensiones de Jakarta EE).

15.1 Paso 1: la entidad JPA y la unidad de persistencia

Igual que en Spring, una entidad JPA representa la tabla. La diferencia está en la configuración: JSF/Jakarta EE usa un archivo `persistence.xml` que define la «unidad de persistencia» y, de forma característica, una fuente de datos gestionada por JTA, lo que delega las transacciones al servidor.

Crear en: `src/main/resources/META-INF/persistence.xml`

persistence.xml

```
1 <persistence xmlns="https://jakarta.ee/xml/ns/persistence"
2     version="3.2">
3   <persistence-unit name="uteqPU" transaction-type="JTA">
4     <jta-data-source>java:/PostgresDS</jta-data-source>
5     <properties>
6       <property name="jakarta.persistence.schema-generation.database.action"
7         value="update"/>
8     </properties>
9   </persistence-unit>
10 </persistence>
```

La entidad `Estudiante` es idéntica a la de Spring (las anotaciones `jakarta.persistence` son el estándar que ambos comparten): `@Entity`, `@Id`, `@GeneratedValue` y sus getters/setters. JPA es común; lo que cambia es el framework que la rodea.

15.2 Paso 2: el bean gestionado (CDI) con JPA y JTA

El «bean gestionado» es el puente entre la vista y los datos. Se anota con `@Named` (para que la vista lo vea) y con un alcance (aquí `@ViewScoped`, que vive mientras el usuario esté en la página). Obtiene el `EntityManager` por inyección (`@PersistenceContext`) y marca como transaccionales los métodos que modifican datos con `@Transactional` (JTA).

Crear en: `src/main/java/com/uteq/bean/EstudianteBean.java`

EstudianteBean.java

```

1 package com.uteq.bean;
2
3 import com.uteq.modelo.Estudiante;
4 import jakarta.annotation.PostConstruct;
5 import jakarta.enterprise.context.RequestScoped;
6 import jakarta.inject.Named;
7 import jakarta.persistence.*;
8 import jakarta.transaction.Transactional;
9 import java.util.List;
10
11 @Named           // visible para la vista como #{estudianteBean}
12 @RequestScoped
13 public class EstudianteBean {
14     @PersistenceContext(unitName = "uteqPU")
15     private EntityManager em;
16
17     private Estudiante estudiante = new Estudiante();
18
19     public List<Estudiante> getList() {
20         return em.createQuery("SELECT e FROM Estudiante e ORDER BY e.id",
21                               Estudiante.class).getResultList();
22     }
23
24     @Transactional // JTA: el contenedor abre y confirma la transaccion
25     public String guardar() {
26         em.persist(estudiante);
27         estudiante = new Estudiante();
28         return "lista?faces-redirect=true"; // patron PRG
29     }
30
31     @Transactional
32     public void eliminar(Estudiante e) {
33         em.remove(em.merge(e));
34     }
35
36     public Estudiante getEstudiante() { return estudiante; }
37     public void setEstudiante(Estudiante e) { this.estudiante = e; }
38 }

```

Qué hace JTA por ti

Con `@Transactional`, el contenedor inicia una transacción antes del método y la confirma al terminar (o la deshace si hay un error). No llamas a `commit` ni a `rollback`: es la versión gestionada de las transacciones que en la Parte I controlabas a mano con JDBC.

15.3 Paso 3: la vista con componentes (Facelets)

La vista es un archivo `.xhtml` que usa componentes JSF (prefijo `h:`). El componente `h:dataTable` recorre la lista del bean; `h:form` e `h:inputText` construyen el formulario; y `h:commandButton` invoca un método del bean al pulsarse. La conexión vista-bean se escribe con EL, igual que en la JSP,

pero con `#{...}`.

Crear en: `src/main/webapp/lista.xhtml`

lista.xhtml

```

1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml"
3     xmlns:h="jakarta.faces.html">
4 <h:head><title>Estudiantes</title></h:head>
5 <h:body>
6     <h:form>
7         <h:dataTable value="#{estudianteBean.lista}" var="e" border="1">
8             <h:column><f:facet name="header">Nombre</f:facet>
9                 #{e.nombre}</h:column>
10            <h:column><f:facet name="header">Carrera</f:facet>
11                #{e.carrera}</h:column>
12            <h:column>
13                <h:commandButton value="Eliminar"
14                    action="#{estudianteBean.eliminar(e)}"/>
15            </h:column>
16        </h:dataTable>
17    </h:form>
18
19    <h:form>
20        <h:inputText value="#{estudianteBean.estudiante.nombre}"/>
21        <h:inputText value="#{estudianteBean.estudiante.correo}"/>
22        <h:inputText value="#{estudianteBean.estudiante.carrera}"/>
23        <h:commandButton value="Guardar" action="#{estudianteBean.guardar}"/>
24    </h:form>
25 </h:body>
26 </html>

```

15.4 Paso 4: inicio de sesión y control de acceso en JSF

En Jakarta EE el control de acceso puede declararse en la configuración de seguridad del servidor (seguridad declarativa) o gestionarse con un bean de sesión y un filtro, de forma análoga a la Parte I. Un patrón sencillo y didáctico es un bean de sesión que guarda al usuario autenticado, consultable desde las vistas y desde un filtro que protege las páginas.

Crear en: `src/main/java/com/uteq/bean/SesionBean.java`

SesionBean.java

```

1 package com.uteq.bean;
2
3 import jakarta.enterprise.context.SessionScoped;
4 import jakarta.inject.Named;
5 import java.io.Serializable;
6
7 @Named
8 @SessionScoped
9 public class SesionBean implements Serializable {
10     private String usuario; // null si no ha iniciado sesion
11
12     public boolean isAuthenticated() { return usuario != null; }
13     public String getUsuario() { return usuario; }
14     public void setUsuario(String u) { this.usuario = u; }
15
16     public String cerrarSesion() {
17         usuario = null;
18         return "login?faces-redirect=true";
19     }
20 }

```

Tras un inicio de sesión válido (un bean de login que verifica las credenciales con BCrypt y hace `sesionBean.setUsuario(...)`), las vistas pueden mostrar u ocultar opciones con `rendered="#{sesionBean.autenticado}"`, y un filtro puede redirigir al login a quien intente entrar sin estar autenticado, exactamente con la misma idea que viste en la Parte I.

16 Cierre: los tres enfoques comparados

Has resuelto el mismo problema de tres maneras. Conviene mirarlas juntas para entender qué aporta cada una y cuándo elegirla. Ninguna es «la mejor»: dependen del contexto, del equipo y del tipo de proyecto.

Enfoque	Características y cuándo usarlo
Servlets + JSP	Tecnología base; máximo control y comprensión. Ideal para aprender y para aplicaciones pequeñas.
Spring Boot	Autoconfiguración, inyección, Spring Data y Security. Muy productivo; estándar de facto en la industria.
Jakarta Faces	Basado en componentes, dentro de Jakarta EE, con JPA y JTA. Fuerte en entornos empresariales con servidor de aplicaciones.

Lo esencial es común a los tres: el modelo de petición–respuesta, la separación en capas, el acceso a datos seguro con sentencias preparadas o un ORM, la sesión para el estado y un mecanismo de control de acceso. Si dominas esos conceptos —y al llegar aquí los dominas—, cambiar de tecnología es solo cuestión de aprender su sintaxis.

16.1 Recomendación de aprendizaje

Empieza por entender bien la Parte I: te da los cimientos. Para proyectos nuevos en la industria, Spring Boot es hoy la opción más demandada y productiva. Jakarta Faces es valioso si trabajarás en entornos empresariales basados en el estándar Jakarta EE. En todos los casos, los principios de diseño —capas, MVC, seguridad— son los que marcan la diferencia entre un código que funciona y uno que perdura.

16.2 Glosario

Términos y siglas que aparecen en la guía, para consulta rápida.

Término	Significado
HTTP	Protocolo de petición–respuesta de la web.
Servlet	Clase Java que atiende peticiones HTTP (el controlador).
JSP	JavaServer Pages: vista basada en HTML con datos dinámicos.
EL	Expression Language: acceso a datos en la vista con <code>\${...}</code> .
JSTL	Biblioteca de etiquetas para lógica de presentación.
MVC	Modelo–Vista–Controlador (patrón de separación).
DAO	Data Access Object: capa de acceso a datos.
Filtro	Componente que intercepta peticiones (p. ej. control de acceso).
Sesión	Estado del usuario en el servidor entre peticiones.
WAR	Web Application Archive: empaquetado de la aplicación.
ORM	Mapeo objeto-relacional (objetos en vez de SQL).
JPA	Jakarta Persistence: el estándar de ORM en Java.
JTA	Jakarta Transactions: transacciones gestionadas por el contenedor.
CDI	Contexts and Dependency Injection (inyección en Jakarta EE).
Thymeleaf	Motor de plantillas habitual en Spring Boot.

Término	Significado
Facelets	Vistas .xhtml basadas en componentes en JSF.

16.3 Recursos

Para profundizar, estos son los recursos de referencia más fiables.

- Jakarta EE (servlets, JSP, JSF, JPA): **jakarta.ee**
- Apache Tomcat: **tomcat.apache.org**
- Spring Boot: **spring.io/projects/spring-boot**
- Spring Security: **spring.io/projects/spring-security**
- Jakarta Faces: **jakarta.ee/specifications/faces**
- PostgreSQL: **postgresql.org/docs**

Con esta guía has recorrido el desarrollo web con Java de extremo a extremo: de los cimientos —servlets, JSP, JSTL y EL— a los frameworks que los potencian. Cada problema que planteamos tiene ahora una solución que entiendes a fondo, y el problema completo está resuelto de tres formas distintas.